PRUEBA TÉCNICA · DÍA DE LA PRUEBA

Pasos y comandos para levantar todo

Del git clone al login funcionando: Oracle en Docker, backend Quarkus o Spring Boot, Angular y pruebas en Swagger. Cada paso indica cómo saber que funcionó y qué hacer si falla. Al final, un resumen de una página.

Java 21 · Oracle · Angular 22 · JWT · Spring Boot · Quarkus

Proyecto: `login-jwt` (login con JWT, roles ADMIN y VIEW, CRUD sobre Oracle)

Pasos para el día de la prueba

Sigue el orden. Cada paso dice **cómo saber que funcionó** antes de pasar al siguiente.

0. Revisar la máquina (2 min)

```
java -version        # 17 o superior (ideal 21)
node -v              # 20 o superior
docker -v            # Docker instalado
git --version
```

> No hace falta Maven instalado: cada backend trae `mvnw.cmd` (Maven Wrapper). No hace falta Angular CLI global: se usa `npx ng ...`.

Sobre `.\mvnw` y `.\mvnw.cmd` en PowerShell: los dos funcionan. PowerShell completa la extensión con la variable `PATHEXT` y encuentra `mvnw.cmd`; el archivo `mvnw` sin extensión es para Linux, Mac o Git Bash. Lo que **no** se puede omitir es el `.\`: PowerShell no ejecuta programas de la carpeta actual sin ruta explícita, por seguridad.

Terminal	Comando
PowerShell	`.\mvnw quarkus:dev`
CMD	`mvnw quarkus:dev`
Git Bash	`./mvnw quarkus:dev`

1. Bajar el proyecto

```
cd C:\
git clone <URL-de-tu-repo> login-jwt
cd C:\login-jwt
```

Usa una ruta corta, sin espacios y fuera de OneDrive o el Escritorio.

2. Oracle en Docker

Opción A: ellos te dan el Oracle dockerizado

No uses `docker-compose.yml`. Pide **host, puerto, service name, usuario y clave** y:

1. Ejecuta `db/01_schema.sql` en su base (DBeaver/SQL Developer, desde `CREATE TABLE` hacia abajo, sin la línea `CONNECT`).
2. Cambia esas 4 líneas en `application.properties` del backend que uses.

Opción B: usar el nuestro

```
docker compose up -d
docker compose ps          # repetir hasta ver (healthy). La 1a vez: 1 a 3 minutos
docker compose logs -f     # opcional: ver el avance. Ctrl+C para salir
```

Funcionó si: en los logs aparece `DATABASE IS READY TO USE!` y se ejecutó `01_schema.sql`.

Problema	Solución
`port is already allocated` (1521 ocupado)	Otro Oracle corriendo: `docker ps` y `docker stop <nombre>`. En tu PC de práctica: `docker stop oracle-prep`
Cambié el `.sql` y no se aplica	Los scripts solo corren la 1a vez: `docker compose down -v` y `up -d` de nuevo

Comprobar por consola (opcional):

```
docker exec -it oracle-login sqlplus app/app123@//localhost/FREEPDB1
SQL> SELECT username, rol FROM usuario;
SQL> exit
```

3. Backend (elige UNO)

Quarkus (puerto 8081)

```
cd C:\login-jwt\backend-quarkus
.\mvnw.cmd quarkus:dev
```

Funcionó si: ves `Listening on: http://localhost:8081`. Abre http://localhost:8081/q/swagger-ui

Spring Boot (puerto 8080)

```
cd C:\login-jwt\backend-spring
.\mvnw.cmd spring-boot:run
```

Funcionó si: ves `Tomcat started on port 8080`. Abre http://localhost:8080/swagger-ui.html

> La primera vez Maven descarga dependencias: varios minutos. Necesita internet.

Probar el login en Swagger

1. `POST /api/auth/login` → *Try it out* → body `{"username":"admin","password":"admin123"}` → *Execute*.
2. Copia el `token` de la respuesta (sin comillas).
3. Botón **Authorize** (candado, arriba a la derecha) → pega el token → *Authorize*.
4. Ya puedes probar `GET /api/contribuyentes`, `POST`, etc.

Con `viewer` el `POST` debe dar **403**. Sin token, cualquier `/api/contribuyentes` da **401**.

Error en la consola	Significa
`ORA-12541` / `Connection refused`	Oracle apagado o puerto equivocado
`ORA-12514`	Service name incorrecto (`FREEPDB1`)
`ORA-01017`	Usuario o clave de Oracle incorrectos
`ORA-00942: table or view does not exist`	No se ejecutó `01_schema.sql` en ese usuario
`Port 8081 already in use`	Ya hay otro backend corriendo: ciérralo

4. Frontend Angular

```
cd C:\login-jwt\frontend
npm install            # solo la primera vez (node_modules no se sube a git)
npx ng serve
```

1. Abre `src/app/api.config.ts` y deja la URL del backend que levantaste: `ts export const API_URL = 'http://localhost:8081'; // Quarkus · Spring: 8080`
2. Abre http://localhost:4200 → te manda a `/login`.

Funcionó si: con `admin/admin123` ves la tabla y el formulario; con `viewer/viewer123` solo la tabla.

Mensaje en pantalla	Causa
"No se pudo conectar con el servidor..."	Backend apagado, URL incorrecta en `api.config.ts` o CORS
"Usuario o contraseña incorrectos"	Es lo esperado con una clave mala (401)

Para ver el detalle de cualquier error: **F12 → pestaña Red (Network)**, clic en la petición roja.

5. Si piden cambios típicos

Te piden...	Toca...
Un campo nuevo en contribuyente	`01_schema.sql` (ALTER TABLE) → entidad → `ContribuyenteRequest` → `contribuyente.service.ts` → `contribuyentes.html`
Que VIEW también pueda crear	Quarkus: `@RolesAllowed({"ADMIN","VIEW"})` en `crear` · Spring: regla en `SecurityConfig`
Un tercer rol	`CHECK` de `usuario.rol` en SQL + las reglas de arriba + `esAdmin` en `auth.service.ts`
Cambiar la duración del token	`app.jwt.duracion-segundos` en `application.properties`
Un usuario nuevo	Generar hash BCrypt (ver `GUIA-JWT.md`) e `INSERT INTO usuario`

6. Subir tus cambios (si te lo piden)

```
cd C:\login-jwt
git status
git add .
git commit -m "Prueba técnica: login JWT y CRUD por roles"
git push
```

Plan B: crear los proyectos desde cero (si no puedes usar el repo)

Quarkus: code.quarkus.io

Enlace con todo ya marcado:

https://code.quarkus.io/?g=com.hacienda&a=backend-quarkus&j=21&e=rest-jackson&e=hibernate-orm-panache&e=jdbc-oracle&e=hibernate-validator&e=smallrye-openapi&e=smallrye-jwt&e=smallrye-jwt-build

Extensión (buscar por este nombre)	Para qué
REST Jackson	Endpoints con JSON
Hibernate ORM with Panache	ORM
JDBC Driver - Oracle	Driver
Hibernate Validator	`@NotBlank` , `@Size`
SmallRye OpenAPI	Swagger
SmallRye JWT	**Valida** el token (`@RolesAllowed`)
SmallRye JWT Build	**Genera** el token (`Jwt.issuer(...).sign()`)

⚠ `BcryptUtil` **no aparece en la página** porque es una librería, no una extensión. Agrégala a mano en `pom.xml` , dentro de `<dependencies>` :

```
<dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-elytron-security-common</artifactId>
</dependency>
```

Llaves RSA para firmar el token, dentro de `src\main\resources` :

```
cd src\main\resources
# Si "openssl" no se reconoce, usa la ruta de Git: & "C:\Program Files\Git\usr\bin\openssl.exe"
openssl genrsa -out rsa.pem 2048
openssl pkcs8 -topk8 -nocrypt -in rsa.pem -out privateKey.pem
openssl rsa -in rsa.pem -pubout -out publicKey.pem
del rsa.pem
```

Spring Boot: start.spring.io

Enlace con todo ya marcado:

https://start.spring.io/#!type=maven-project&language=java&packaging=jar&jvmVersion=21&groupId=com.hacienda&artifactId=backend-spring&name=backend-spring&packageName=com.hacienda&dependencies=web,data-jpa,validation,security,oauth2-resource-server,oracle

Revisa que en *Spring Boot* esté elegida una versión **sin** `SNAPSHOT` ni `RC` (la estable, por ejemplo `4.1.2`).

Dependencia (buscar con **ADD DEPENDENCIES**)	Para qué
Spring Web	Endpoints REST
Spring Data JPA	ORM + repositorios
Validation	`@NotBlank` , `@Size`
Spring Security	Seguridad + BCrypt
OAuth2 Resource Server	**Valida** el JWT y trae `JwtEncoder` para **generarlo**
Oracle Driver	Driver

⚠ **Swagger (springdoc) no está en la página.** Agrégalo a mano en `pom.xml` :

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>3.2.0</version>
</dependency>
```

Spring no necesita llaves RSA: usa la clave secreta `app.jwt.secret` del `application.properties`.

Después de generar cualquiera de los dos

1. **GENERATE** → descomprimir dentro de `C:\login-jwt\` → revisar que no quede `backend\backend\pom.xml`.
2. Borrar los archivos de ejemplo (`GreetingResource`, `MyEntity`, sus tests).
3. Rehacer `application.properties` y los paquetes `auth` y `contribuyente` siguiendo la estructura del PDF **Guía JWT y mapa del código** (ahí está qué archivo va y qué hace cada uno, no el código completo).
4. Levantar con `.\mvnw quarkus:dev` o `.\mvnw spring-boot:run`.

7. Apoyo extra: Claude en la terminal de otra PC

> ⚠ **Antes de usarlo, pregunta si está permitido.** Si la regla es "solo Gemini", respétala.

```
irm https://claude.ai/install.ps1 | iex      # o: npm install -g @anthropic-ai/claude-code
claude
```

1. Al iniciar sesión, la terminal muestra un **enlace**.
2. Ábrelo **en tu celular**, donde ya tienes tu sesión de Claude, y autoriza.
3. Copia el código que te muestra y pégalo en la terminal. No hace falta abrir tu correo en esa PC.
4. **Al terminar:** escribe `/logout` para no dejar tu cuenta abierta en una máquina ajena.

Resumen de una página

#	Qué	Comando	Listo cuando…
0	Revisar herramientas	`java -version` · `node -v` · `docker -v`	Java 17+, Node 20+
1	Bajar el proyecto	`git clone <url> C:\login-jwt`	Existe la carpeta
2	Oracle	`docker compose up -d` · `docker compose ps`	`(healthy)`
3a	Quarkus	`cd backend-quarkus` · `.\mvnw quarkus:dev`	`Listening on: http://localhost:8081`
3b	Spring	`cd backend-spring` · `.\mvnw spring-boot:run`	`Tomcat started on port 8080`
4	Angular	`cd frontend` · `npm install` · `npx ng serve`	http://localhost:4200 muestra el login
5	Apuntar Angular	editar `src/app/api.config.ts`	`admin/admin123` ve la tabla
6	Probar la API	Swagger → login → **Authorize**	`viewer` en POST da 403